

Universidad de Valladolid

Práctica 2: Pruebas de carga, Locust y Prometheus

Grado en Ingeniería Informática
Evaluación de Sistemas Informáticos

Grupo 16

Índice

1. Introducción **3**
1.1. Descripción del sistema bajo prueba . . . 3

2. Estructura del proyecto **3**

3. Prometheus **4**
3.0.1. Funciones/Métricas que vamos a emplear en la práctica . . . 5

4. Definición de las métricas que vamos a recoger **6**
4.1. Sintaxis de los contadores . . . 6
4.2. Exposición del endpoint /metrics — app.py . . . 10
4.3. Instrumentación de flujos OAuth2 — oauth2_routes.py . . . 13
4.4. Verificación del sistema de métricas . . . 14

5. Despliegue de Prometheus **14**
5.1. Instalación . . . 14
5.2. Configuración . . . 14
5.3. Arranque . . . 15

6. Conexión con Grafana **15**
6.1. Arquitectura del sistema . . . 15
6.2. Configuración del Data Source . . . 16

7. Locust **16**

8. Enfoque EDR del sistema de métricas **28**
8.1. Motivación . . . 28
8.2. Métricas adicionales implementadas . . . 28
8.2.1. Categoría 1 — Rendimiento del sistema . . . 29
8.2.2. Categoría 2 — Estado de tokens . . . 29
8.2.3. Categoría 3 — Detección de ataques . . . 29

9. Modificaciones a metrics.py **30**

10. Modificaciones a models.py **30**
10.1. store_authorization_code . . . 30
10.2. consume_authorization_code . . . 30
10.3. issue_token_pair . . . 31
10.4. validate_refresh_token . . . 31
10.5. revoke_refresh_token . . . 32

11. Modificaciones a app.py **32**
11.1. Exposición del endpoint /metrics . . . 32
11.2. Registro automático de métricas HTTP . . . 33

12. Pruebas de carga con Locust **34**
12.1. Instalación . . . 34
12.2. Diseño del locustfile . . . 34
12.3. Descripción de cada tarea . . . 34
12.4. Configuración de las pruebas . . . 35

12.5. Protección de la VM durante las pruebas 35

13. Verificación manual de métricas **36**

14. Grafana **38**
14.1. Ataques de fuerza burta vs *credential stuffing* 39
14.2. Ratio de Tokens 39
14.3. Fallos del login de los usuarios 40
14.4. Fallos del login de los usuarios 40
14.5. Logins correctos en el sistema. 41
14.6. Peticiones a los diferentes endpoints de nuestro sistema 41
14.7. Distribución de las peticiones en nuestro sistema 42
14.8. Ratio peticiones correctas fallidas por los usuarios 42
14.8.1. Tiempo de respuesta a cada *endpint* 43
14.8.2. Tipos de *tokens* en nuestro sistema. 43
14.8.3. Distribución de *tokens* en el sistema 44
14.8.4. Intentos de ataque de fuerza bruta 44
14.8.5. Flujos de autenticació en cada uno de los *endpoints* 45

1 Introducción

Para esta práctica estaremos empleando un servidor de autenticación *OAuth2*, el cual es el servidor por excelencia para hacer de mediación en la tarea de distribución de claves de acceso sobre el que realizaremos pruebas de carga con Locust, el cual es un sistema de monitorización que aplica el modelo *pull*, el cual, está apuntando a nuestro *endpoint* y cada 15 segundos (según lo definido en su *.yml*) anota todo lo que está pasando, estos eventos serán visualizados posteriomente en Grafana.
De tal manera que tenemos 3 objetivos:

1. Instrumentación del backend con métricas Prometheus (haremos incapie en esta tecnología en 3).
2. Despliegue y configuración de Prometheus y Grafana.
3. Pruebas de carga y estrés con Locust.

1.1 Descripción del sistema bajo prueba

El backend proporcionado es un servidor de autenticación OAuth2 desarrollado en Python con Flask y SQLite, que implementa los tres flujos principales definidos en el RFC 6749:

- **Authorization Code**: flujo orientado a aplicaciones web donde un usuario humano se autentica mediante un formulario de login. El servidor emite un código temporal que posteriormente se canjea por un par de tokens.
- **Client Credentials**: flujo orientado a aplicaciones máquina a máquina donde no hay un usuario humano. La aplicación se autentica directamente con sus credenciales y obtiene un token de acceso.
- **Refresh Token**: flujo que permite renovar un token de acceso caducado sin necesidad de repetir el proceso de autenticación completo.

La aplicación se gestiona con `uv`, el cual ya empleamos en nuestra primera práctica y como se explicó en el informe es un gestor de paquetes para Python más moderno que el típico pip, y se ejecuta en el puerto 5001.

2 Estructura del proyecto

Para entender el proyecto vamos a ver qué hacen los ficheros que han sido proporcionados por el profesor y los que hemos tenido que crear para el mismo:

Fichero	Descripción
`app.py`	Punto de entrada de la aplicación Flask. Registra blueprints, configura la base de datos y expone el endpoint `/metrics`.
`oauth2_routes.py`	Implementación de los endpoints OAuth2: `/authorize` y `/token`.
`admin_routes.py`	Endpoints de administración bajo `/admin/clients` para gestionar clientes OAuth2.
`models.py`	Capa de acceso a datos: clientes, códigos de autorización y tokens.
`database.py`	Gestión de la conexión SQLite y definición del esquema.
`metrics.py`	Definición centralizada de todas las métricas Prometheus (fichero creado en esta práctica).
`seed.py`	Script de inicialización que crea dos clientes de prueba en la base de datos.

Cuadro 1: Estructura de ficheros del proyecto

3 Prometheus

Prometheus es una base de datos de series temporales de tipo *pull*, esto quiere decir que es la propia base de datos quien periódicamente consulta a las aplicaciones por sus métricas, de tal manera que las aplicaciones toman un papel pasivo, donde solo se encagan de realizar sus tareas y sólo tiene que exponer un *enpoint* llamado */metrics* con los datos en el mismo formato que emplea esta tecnología. Para lograr esto vamos a emplear una librería de Python, `prometheus_client` la cual se encarga de gestionar este endpoint y formatear los datos automáticamente.

```
uv add prometheus-client
```

Aquí uv juega un papel fundamental, puesto que de *add* de manera automática instala la librería, actualiza el .toml del proeycto y registra la dependencia en el *uv.lock*.
Para ello hemos tenido que modificar el fichero de prometheus de la siguiente manera.

```
global:
  #intervalo de busqueda de las metricas en el localhost 5001
  scrape_interval: 15s
  #intervalo de evaluacion de las reglas, para que nos llame grafana si se
     lia
  #no lo vamos a emplear en la practica
  evaluation_interval: 15s

scrape_configs:
  - job_name: "prometheus"
    static_configs:
      - targets: ["localhost:9090"]

  #Esto es un nuevo trabajo que le dice al prometheus donde tiene que ir a
     buscar
```

```
  - job_name: "oauth2"
    static_configs:
      - targets: ["localhost:5001"]
```

3.0.1. Funciones/Métricas que vamos a emplear en la práctica

Emplearemos 2 de las 4 métricas que nos proporciona la apliación, puesto que son las que más se adaptan al planteamiento que tenemos para la práctica:

- **Counter**: contador de incremento monotónico. Solo puede crecer. Se usa para contar eventos como peticiones recibidas, tokens emitidos o errores producidos. Por convención, los counters tienen el sufijo `_total`.
- **Gauge**: contador que puede crecer y decrecer. Se usa para valores que fluctúan como memoria usada o conexiones activas.

4 Definición de las métricas que vamos a recoger

Hemos creado un fichero *metrics.py* en el que hemos creado todos los contadores, con dos objetivos, por una parte centralizarlo todo en un fichero, facilitando el desarrollo y la depuración del código y por otra parte, evitar las importaciones circulares de módulos.
Este fenómeno ocurre cuando dos ficheros se importan entre ellos, es decir, si por ejemplo en *oauth2_metrics.py* importamos cosas de *app.py* y en el último importamos fucniones del primero, esto lo solucionamos gestionando todos los contadores en un mismo fichero.

4.1 Sintaxis de los contadores

Antes de mostrar todos los contadores, vamos a explicar un poco la sintaxis de este fichero, puesto que es una tecnología nueva, que al menos en la carrera no hemos tocado mucho:

- auth_flows_started = Counter(: nombre de variable la cual es un contador, la cual sólo puede incrementarse
- .ºauth2_auth_flows_started_total": nombre de la métrica para su posterior integración con grafana
- "Flujos de autenticacion iniciados", : descripción de la metrica
- ["*grant_type*"]: la etiqueta para luego poder filtrarlo en el grafana.)

```
#Codigo para recopilar las metricas de Prometheus.

from prometheus_client import (
    Counter,
    Histogram,
    Gauge
)

#Flujos de autenticacion
# Contabiliza los intentos de llamada al endpoint de authorized o de token (exitosos o no)
auth_flows_started = Counter(
    "oauth2_auth_flows_started_total",
    "Flujos de autenticación iniciados",
    ["grant_type"] #permite almacenar el resultado de cada contador de manera
        independiente para poder tener una cifra de cada uno.
)

#Contabiliza los intentos exitosos de autenticacion en el sistema
auth_flows_success = Counter(
    "oauth2_auth_flows_success_total",
    "Flujos de autenticación completados con éxito",
    ["grant_type"]
)

#Contabiliza los intentos fallidos de autenticacion en el sistem
auth_flows_failed = Counter(
```

```
    "oauth2_auth_flows_failed_total",
    "Flujos de autenticación fallidos",
    ["grant_type"]
)

#Tokens emitidos por la aplicacion a nivel general
tokens_issued = Counter(
    "oauth2_tokens_issued_total",
    "Tokens emitidos",
    ["token_type", "grant_type"]
)

#Tokens emitidos (separados por tipo)
access_tokens_issued = Counter(
    "oauth2_access_tokens_issued_total",
    "Access tokens emitidos",
    ["grant_type"]
)
refresh_tokens_issued = Counter(
    "oauth2_refresh_tokens_issued_total",
    "Refresh tokens emitidos",
    ["grant_type"]
)

#Llamadas HTTP para medir el numero de peticiones a nivel general
http_requests = Counter(
    "oauth2_http_requests_total",
    "Número de llamadas HTTP",
    ["method", "endpoint", "status_code"]
)

#Llamadas HTTP (separadas por endpoint)
http_requests_token = Counter(
    "oauth2_http_requests_token_total",
    "Llamadas al endpoint /token",
    ["method", "status_code"]
)

http_requests_authorize = Counter(
    "oauth2_http_requests_authorize_total",
    "Llamadas al endpoint /authorize",
    ["method", "status_code"]
)

http_requests_admin = Counter(
    "oauth2_http_requests_admin_total",
    "Llamadas a los endpoints /admin",
    ["method", "status_code"]
)

#Permite la medicion del numero de request que estan en proceso en este momento
requests_in_progress = Gauge(
    "oauth2_requests_in_progress",
```

```
    "Peticiones HTTP en curso en este momento",
    ["endpoint"]
)

#Metricas extra para el planteamiento de ser un EDR
#Con esto podemos medir el tiempo medio de duracion de un request
request_duration = Histogram(
    "oauth2_request_duration_seconds",
    "Tiempo de respuesta de cada peticion",
    ["endpoint", "method"],
    # 0.25 es el umbral de 250ms que marca el enunciado como referencia
    buckets=[0.01, 0.05, 0.1, 0.25, 0.5, 1.0, 2.5, 5.0]
)

#Con esto intentamos ver si esta pasando algo raro con los tokens
#Info sobre los tokens activos en el sistema
active_tokens = Gauge(
    "oauth2_active_tokens",
    "Tokens de acceso activos en el sistema",
    ["token_type"]
)

#Info sobre los tokens caducados
tokens_revoked = Counter(
    "oauth2_tokens_revoked_total",
    "Tokens revocados",
    ["token_type"]
)

#Nos da inormacion sobre los codigos emitos pero no empleados
pending_auth_codes = Gauge(
    "oauth2_pending_auth_codes",
    "Codigos de autorizacion emitidos pero no canjeados"
)

# Intentos de uso de tokens ya revocados
# Señal de posible token replay attack
revoked_token_usage = Counter(
    "oauth2_revoked_token_usage_total",
    "Intentos de uso de tokens ya revocados"
)

# Intentos de uso de tokens expirados
expired_token_usage = Counter(
    "oauth2_expired_token_usage_total",
    "Intentos de uso de tokens expirados"
)

# Intentos de reutilizacion de codigos de autorizacion ya usados
# Señal de posible authorization code replay attack
auth_code_reuse = Counter(
    "oauth2_auth_code_reuse_total",
    "Intentos de reutilizacion de codigos ya usados"
```

```
)

# Intentos fallidos de autenticacion por client_id
# Util para detectar ataques de fuerza bruta contra un cliente concreto
brute_force_attempts = Counter(
    "oauth2_brute_force_attempts_total",
    "Intentos fallidos de autenticacion por cliente",
    ["client_id"]
)

# Intentos fallidos de login de usuario
# Util para detectar ataques de fuerza bruta contra usuarios concretos
user_auth_failed = Counter(
    "oauth2_user_auth_failed_total",
    "Intentos fallidos de autenticacion de usuario",
    ["username", "client_id"]
)

# Operaciones de gestion de clientes OAuth2
clients_created = Counter(
    "oauth2_admin_clients_created_total",
    "Clientes OAuth2 creados via API admin"
)

clients_deleted = Counter(
    "oauth2_admin_clients_deleted_total",
    "Clientes OAuth2 eliminados via API admin"
)

clients_listed = Counter(
    "oauth2_admin_clients_listed_total",
    "Consultas al listado de clientes"
)

user_auth_success = Counter(
    "oauth2_user_auth_success_total",
    "Inicios de sesion exitosos por usuario",
    ["username"]
)

#Vamos a simular el comportamiento de un usuarios
#metrica para los clicks del formulario.
login_page_visits = Counter(
    "oauth2_login_page_visits_total",
    "Visitas a la pagina de login por client_id",
    ["client_id"]
)
#Intentos de login con un usario desconocido
unknown_client_attempts = Counter(
    "oauth2_unknown_client_attempts_total",
    "Intentos de acceso con client_id desconocido"
)

```

```
# Un valor alto indica credential stuffing
# Un valor bajo con muchos fallos indica fuerza bruta clasica
unique_failed_usernames = Gauge(
    "oauth2_unique_failed_usernames_active",
    "Numero de usernames distintos con fallos de login activos"
)
```

Las **etiquetas** (*labels*) permiten que un mismo counter almacene valores independientes para cada combinación de etiquetas. Por ejemplo, `auth_flows_started` con la etiqueta `grant_type` almacena un contador separado para cada tipo de flujo:

```
oauth2_auth_flows_started_total{grant_type="
   client_credentials"} 10.0
oauth2_auth_flows_started_total{grant_type="
   authorization_code"} 3.0
oauth2_auth_flows_started_total{grant_type="refresh_token"}
    5.0
```

4.2 Exposición del endpoint /metrics — app.py

Se modifica `app.py` para exponer el endpoint `/metrics` y registrar automáticamente todas las peticiones HTTP mediante decoradores Flask:

```
"""
app.py --Flask application entry point.

Wires together the database, blueprints, and configuration.
Run with: uv run python app.py
"""
#DISCLAIMER we've commented in english to mantain the consistence of the code

#request had the request info
#g will be use to store the request live time
from flask import Flask, request, g
#wsi will be the way promethus get their data
from prometheus_client import make_wsgi_app
#use for the comunication btw the wsgi and metrics.py
from werkzeug.middleware.dispatcher import DispatcherMiddleware
import time

from database import close_db, init_db
from oauth2_routes import oauth2
from admin_routes import admin
#we need this lib for the metrics that we've allready defined on metrics.py
from metrics import (
       http_requests,
       http_requests_token,
       http_requests_admin,
```

```
        http_requests_authorize,
        request_duration,
        requests_in_progress,
        unknown_client_attempts
)

from werkzeug.serving import run_simple
# ---------------------------------------------------------------------------
# Hardcoded test users (resource owners for the authorization code flow).
# In production these would live in a proper user store.
# ---------------------------------------------------------------------------
USERS = {
    "testuser": "testpass",
    "admin": "adminpass",
}


def create_app() -> Flask:
    app = Flask(__name__)
    app.secret_key = "dev-secret-key-change-in-production"

    # Store users in app config so routes can access them
    app.config["USERS"] = USERS

    # Register blueprints
    app.register_blueprint(oauth2)
    app.register_blueprint(admin)

    # Database lifecycle
    app.teardown_appcontext(close_db)

    # Ensure tables exist on first launch
    with app.app_context():
        init_db()

    #Automatic HTTP metrics log
    # the Flask decorator
    @app.before_request
    def before_request():
        # Store the moment of the request arrives
        g.start_time = time.time()
        requests_in_progress.labels(endpoint=request.path).inc()
    #This force that every request must execute this methods
    @app.after_request
    def after_request(response):
        # get the request endpoint
        endpoint = request.path

        # we increment the general counter
        http_requests.labels(
            method=request.method,
            endpoint=endpoint,
            status_code=str(response.status_code)
```

```
        ).inc()

        # We increment the specific counter for each endpoint
        if "/token" in endpoint:
            http_requests_token.labels(
                method=request.method,
                status_code=str(response.status_code)
            ).inc()
        elif "/authorize" in endpoint:
            http_requests_authorize.labels(
                method=request.method,
                status_code=str(response.status_code)
            ).inc()
        elif "/admin" in endpoint:
            http_requests_admin.labels(
                method=request.method,
                status_code=str(response.status_code)
            ).inc()
        requests_in_progress.labels(endpoint=request.path).dec()
        # Calc and store the request duration
        duration = time.time() - g.start_time
        request_duration.labels(
            endpoint=request.path,
            method=request.method
        ).observe(duration)
        return response

    return app


if __name__ == "__main__":
    # Cretaion of the Flask app
    flask_app = create_app()

    # Merge Flask with the /metrics (Prometheus) endpoint
    application = DispatcherMiddleware(flask_app, {
        "/metrics": make_wsgi_app()
    })

    print("OAuth2 server running on http://localhost:5001")
    print("Metrics available on http://localhost:5001/metrics")
    print("Endpoints:")
    print(" GET /authorize --Authorization Code flow (browser)")
    print(" POST /token --Token endpoint")
    print(" GET /admin/clients --List all clients")
    print(" POST /admin/clients --Register a new client")
    print(" DELETE /admin/clients/<id> --Delete a client")
    print()

    run_simple("0.0.0.0", 5001, application, use_reloader=True, use_debugger=True)
```

Los **decoradores** `@before_request` y `@after_request` son hooks de Flask que se ejecutan automáticamente antes y después de cada petición HTTP respectivamente. Esto permite registrar todas las peticiones en un único lugar sin modificar cada ruta individualmente, siguiendo el principio DRY (*Don't Repeat Yourself*).

El `DispatcherMiddleware` combina la aplicación Flask con la mini aplicación de Prometheus que genera el endpoint `/metrics`, de modo que todo el tráfico a `/metrics` lo gestiona Prometheus y el resto lo gestiona Flask.

4.3 Instrumentación de flujos OAuth2 — oauth2_routes.py

Se modifica `oauth2_routes.py` para incrementar los contadores en cada uno de los tres flujos de autenticación. La lógica es la misma para los tres: se incrementa `started` al entrar en la función, `failed` en cada punto de fallo y `success` junto con los contadores de tokens al completarse con éxito.

A continuación se muestra el flujo `client_credentials` como ejemplo:

```
        def _handle_client_credentials():
        auth_flows_started.labels(
        grant_type="client_credentials").inc()

        client_id, err = _authenticate_client()
        if err:
        auth_flows_failed.labels(
        grant_type="client_credentials").inc()
        return _error_response("invalid_client", err, 401)

        client = get_client(client_id)
        if client is None or \
        "client_credentials" not in client["grant_types"]:
        auth_flows_failed.labels(
        grant_type="client_credentials").inc()
        return _error_response("unauthorized_client",
        "Client is not authorized for client_credentials")

        scope = request.form.get("scope", "")
        token_response = issue_token_pair(
        client_id=client_id,
        scope=scope,
        user_id=None,
        include_refresh=False,
        )

        auth_flows_success.labels(
        grant_type="client_credentials").inc()
        tokens_issued.labels(
        token_type="access",
        grant_type="client_credentials").inc()
        access_tokens_issued.labels(
        grant_type="client_credentials").inc()
        # client_credentials no emite refresh token (RFC 6749)

        return jsonify(token_response)
```

Listing 1: Fragmento de _handle_client_credentials()

Los flujos `authorization_code` y `refresh_token` siguen la misma lógica pero con más puntos de fallo y emiten además un `refresh_token`, por lo que incrementan cinco contadores en lugar de tres al completarse con éxito.

4.4 Verificación del sistema de métricas

Tras arrancar la aplicación y realizar dos peticiones de prueba (una correcta y una con credenciales incorrectas), se verifica el correcto funcionamiento consultando el endpoint `/metrics`:

```
curl http://localhost:5001/metrics | grep oauth2
```

Los resultados obtenidos son los esperados:

```
oauth2_auth_flows_started_total{grant_type="
   client_credentials"} 2.0
oauth2_auth_flows_success_total{grant_type="
   client_credentials"} 1.0
oauth2_auth_flows_failed_total{grant_type="
   client_credentials"} 1.0
oauth2_tokens_issued_total{grant_type="client_credentials",
        token_type="access"} 1.0
oauth2_http_requests_total{endpoint="/token",
        method="POST",status_code="200"} 1.0
oauth2_http_requests_total{endpoint="/token",
        method="POST",status_code="401"} 1.0
```

Los contadores cuadran perfectamente: 2 flujos iniciados, 1 exitoso (petición correcta) y 1 fallido (credenciales incorrectas). Solo se emitió 1 token de acceso, correspondiente al flujo exitoso. No se emitieron refresh tokens porque el flujo `client_credentials` no los emite según el RFC 6749.

5 Despliegue de Prometheus

5.1 Instalación

Se descarga el binario oficial de Prometheus desde GitHub y se extrae:

```
wget https://github.com/prometheus/prometheus/releases/
   download/\
v3.3.1/prometheus-3.3.1.linux-amd64.tar.gz
tar -xvf prometheus-3.3.1.linux-amd64.tar.gz
cd prometheus-3.3.1.linux-amd64
```

5.2 Configuración

Se edita el fichero `prometheus.yml` para añadir un nuevo job que apunte a nuestra aplicación en el puerto 5001:

```
global:
  #intervalo de busqueda de las metricas en el localhost 5001
  scrape_interval: 15s
  #intervalo de evaluacion de las reglas, para que nos llame grafana si se
     lia
  #no lo vamos a emplear en la practica
  evaluation_interval: 15s

scrape_configs:
  - job_name: "prometheus"
    static_configs:
      - targets: ["localhost:9090"]

  #Esto es un nuevo trabajo que le dice al prometheus donde tiene que ir a
     buscar
  - job_name: "oauth2"
    static_configs:
      - targets: ["localhost:5001"]
```

Listing 2: Configuración de Prometheus

El parámetro `scrape_interval` define cada cuánto Prometheus va a buscar métricas a la aplicación. El parámetro `evaluation_interval` define cada cuánto evalúa las reglas de alertas, que no se utilizan en esta práctica.

5.3 Arranque

```
./prometheus --config.file=prometheus.yml
```

Prometheus queda corriendo en el puerto 9090 y empieza a recoger métricas de nuestra aplicación cada 15 segundos.

6 Conexión con Grafana

6.1 Arquitectura del sistema

Una vez desplegados los tres componentes, la arquitectura queda así:

Componente	Puerto	Función
App OAuth2	5001	Genera métricas en `/metrics`
Prometheus	9090	Recoge y almacena las métricas cada 15s
Grafana	3000	Visualiza las métricas en dashboards

Cuadro 2: Arquitectura del sistema de monitorización

El flujo de datos es el siguiente:

`App OAuth2 (5001)` → `Prometheus (9090)` → `Grafana (3000)`

6.2 Configuración del Data Source

Se accede a Grafana en `http://localhost:3000` y se añade Prometheus como fuente de datos en *Connections* >*Data Sources* >*Add data source*. Se configura la URL `http://localhost:9090` y se verifica la conexión con *Save & Test*, obteniendo el mensaje:

"Successfully queried the Prometheus API."

Esto confirma que Grafana puede acceder a los datos almacenados en Prometheus y está listo para visualizarlos en dashboards mediante consultas PromQL.

7 Locust

Una vez que ya teníamos todo configurado vamos a ver como hacemos las pruebas de carga para las métricas.
Planteamos esta primera versión:

```python
from locust import HttpUser, task, between


class OAuth2LoadTest(HttpUser):
    """
    Usuario virtual de Locust para probar el servidor OAuth2.

    Este usuario simula tanto trafico legitimo como trafico erroneo/controlado
    para observar el comportamiento de la aplicacion y sus metricas Prometheus.

    Metricas cubiertas:
        - auth_flows_started / auth_flows_success / auth_flows_failed
        - tokens_issued / access_tokens_issued / refresh_tokens_issued
        - http_requests / http_requests_token / http_requests_authorize /
          http_requests_admin
        - requests_in_progress / request_duration
        - active_tokens / tokens_revoked
        - pending_auth_codes
        - auth_code_reuse
        - revoked_token_usage
        - expired_token_usage (indirectamente via refresh token invalido)
        - brute_force_attempts
        - user_auth_failed
        - active_clients
    """

    wait_time = between(1, 3)

    # Cliente valido para el flujo client_credentials
    VALID_CLIENT_ID = "service-account"
    VALID_CLIENT_SECRET = "6Q6TZj_hKy4xBnZhgBuIwdJ6lSaQbF-9XWR9S_BHthk"
```

```
31
32      # Cliente web para el flujo authorization_code
33      WEB_CLIENT_ID = "web-app"
34      WEB_REDIRECT_URI = "http://localhost:8080/callback"
35
36      # Usuario valido definido en app.py
37      VALID_USERNAME = "testuser"
38      VALID_PASSWORD = "testpass"
39
40      # ============================================================================
41      # FLUJOS LEGITIMOS
42      # ============================================================================
43
44      @task(6)
45      def valid_client_credentials(self):
46          """
47          Trafico legitimo maquina a maquina.
48
49          Metricas activadas:
50              - auth_flows_started{grant_type="client_credentials"}
51              - auth_flows_success{grant_type="client_credentials"}
52              - tokens_issued{token_type="access", grant_type="client_credentials"}
53              - access_tokens_issued{grant_type="client_credentials"}
54              - active_tokens{token_type="access"}
55              - active_clients
56              - http_requests / http_requests_token con status 200
57              - requests_in_progress / request_duration
58          """
59          response = self.client.post("/token", data={
60              "grant_type": "client_credentials",
61              "client_id": self.VALID_CLIENT_ID,
62              "client_secret": self.VALID_CLIENT_SECRET,
63              "scope": "read"
64          })
65
66          if response.status_code != 200:
67              response.failure(f"client_credentials valido fallo con {response.status_code}")
68
69      @task(3)
70      def valid_authorization_code_flow(self):
71          """
72          Flujo completo authorization_code: login -> obtener code -> canjear por tokens.
73
74          Metricas activadas:
75              - auth_flows_started{grant_type="authorization_code"}
76              - auth_flows_success{grant_type="authorization_code"}
77              - tokens_issued{token_type="access"/"refresh", grant_type="authorization_code"}
78              - access_tokens_issued{grant_type="authorization_code"}
79              - refresh_tokens_issued{grant_type="authorization_code"}
80              - active_tokens{token_type="access"} / active_tokens{token_type="refresh"}
81              - pending_auth_codes (sube al emitir el code, baja al canjearlo)
82              - active_clients
83              - http_requests / http_requests_authorize / http_requests_token con status 200
```

```
            - requests_in_progress / request_duration
        """
        # Paso 1: POST /authorize con credenciales validas -> obtenemos el code
        with self.client.post("/authorize", data={
            "client_id": self.WEB_CLIENT_ID,
            "redirect_uri": self.WEB_REDIRECT_URI,
            "response_type": "code",
            "scope": "read",
            "state": "locust-test",
            "username": self.VALID_USERNAME,
            "password": self.VALID_PASSWORD
        }, catch_response=True, allow_redirects=False) as auth_response:

            # El servidor devuelve 302 con el code en la URL de redireccion
            if auth_response.status_code not in (200, 302):
                auth_response.failure(
                    f"authorize valido fallo con {auth_response.status_code}"
                )
                return

            # Extraemos el code del cuerpo o de la cabecera Location segun impl.
            code = None

            location = auth_response.headers.get("Location", "")
            if "code=" in location:
                code = location.split("code=")[1].split("&")[0]

            if not code and auth_response.status_code == 200:
                try:
                    data = auth_response.json()
                    code = data.get("code")
                except Exception:
                    pass

            if not code:
                auth_response.success() # El flujo puede ser distinto, no falla el test
                return

            auth_response.success()

        # Paso 2: POST /token canjeando el code -> obtenemos access + refresh token
        with self.client.post("/token", data={
            "grant_type": "authorization_code",
            "client_id": self.WEB_CLIENT_ID,
            "code": code,
            "redirect_uri": self.WEB_REDIRECT_URI
        }, catch_response=True) as token_response:

            if token_response.status_code != 200:
                token_response.failure(
                    f"canje de code fallo con {token_response.status_code}"
                )
                return
```

```
137
138             token_response.success()
139
140     @task(2)
141     def valid_refresh_token_flow(self):
142         """
143         Flujo completo refresh_token: primero obtiene tokens via authorization_code,
144         luego los renueva con el refresh token.
145
146         Metricas activadas:
147             - auth_flows_started / auth_flows_success {grant_type="refresh_token"}
148             - tokens_issued / access_tokens_issued / refresh_tokens_issued
149             - active_tokens (los nuevos tokens se activan)
150             - tokens_revoked (el refresh token viejo se revoca al rotar)
151             - http_requests_token con status 200
152         """
153         # Paso 1: Obtenemos un par de tokens via authorization_code
154         code = self._get_authorization_code()
155         if not code:
156             return
157
158         with self.client.post("/token", data={
159             "grant_type": "authorization_code",
160             "client_id": self.WEB_CLIENT_ID,
161             "code": code,
162             "redirect_uri": self.WEB_REDIRECT_URI
163         }, catch_response=True) as token_response:
164
165             if token_response.status_code != 200:
166                 token_response.failure(
167                     f"obtencion de tokens para refresh fallo con {token_response.status_code
                        }"
168                 )
169                 return
170
171             try:
172                 tokens = token_response.json()
173                 refresh_token = tokens.get("refresh_token")
174             except Exception:
175                 token_response.failure("No se pudo parsear la respuesta de tokens")
176                 return
177
178             token_response.success()
179
180         if not refresh_token:
181             return
182
183         # Paso 2: Renovamos usando el refresh token
184         with self.client.post("/token", data={
185             "grant_type": "refresh_token",
186             "client_id": self.WEB_CLIENT_ID,
187             "refresh_token": refresh_token
188         }, catch_response=True) as refresh_response:
```

```
189
190             if refresh_response.status_code != 200:
191                 refresh_response.failure(
192                     f"refresh_token valido fallo con {refresh_response.status_code}"
193                 )
194                 return
195
196             refresh_response.success()
197
198     # ================================================================================
199     # FLUJOS DE ERROR / TRAFICO MALICIOSO
200     # ================================================================================
201
202     @task(2)
203     def invalid_client_credentials(self):
204         """
205         Trafico malicioso: client_secret incorrecto.
206
207         Metricas activadas:
208             - auth_flows_started / auth_flows_failed {grant_type="client_credentials"}
209             - brute_force_attempts{client_id="service-account"}
210             - http_requests / http_requests_token con status 401
211         """
212         with self.client.post("/token", data={
213             "grant_type": "client_credentials",
214             "client_id": self.VALID_CLIENT_ID,
215             "client_secret": "secret_incorrecto",
216             "scope": "read"
217         }, catch_response=True) as response:
218
219             if response.status_code == 401:
220                 response.success()
221             else:
222                 response.failure(
223                     f"Se esperaba 401 para credenciales incorrectas, pero llego {response.
                        status_code}"
224                 )
225
226     @task(2)
227     def invalid_user_login(self):
228         """
229         Fallo de login de usuario: password incorrecta.
230
231         Metricas activadas:
232             - auth_flows_started / auth_flows_failed {grant_type="authorization_code"}
233             - user_auth_failed{username="testuser", client_id="web-app"}
234             - http_requests / http_requests_authorize con status 401
235         """
236         with self.client.post("/authorize", data={
237             "client_id": self.WEB_CLIENT_ID,
238             "redirect_uri": self.WEB_REDIRECT_URI,
239             "response_type": "code",
240             "scope": "read",
```

```
            "state": "locust-test",
            "username": self.VALID_USERNAME,
            "password": "password_incorrecta"
        }, catch_response=True) as response:

            if response.status_code == 401:
                response.success()
            else:
                response.failure(
                    f"Se esperaba 401 en login incorrecto, pero llego {response.status_code}
                        "
                )

    @task(1)
    def auth_code_replay_attack(self):
        """
        Intenta canjear el mismo authorization_code dos veces.
        La primera llamada deberia tener exito (200), la segunda debe fallar (400)
        porque el code ya fue marcado como 'used' en la base de datos.

        Metricas activadas:
            - pending_auth_codes (sube al emitir, baja al primer canje)
            - auth_code_reuse (se incrementa en el segundo intento)
            - http_requests / http_requests_token con status 200 y luego 400
        """
        # Obtenemos un code valido
        code = self._get_authorization_code()
        if not code:
            return

        # Primer canje: deberia funcionar (200)
        with self.client.post("/token", data={
            "grant_type": "authorization_code",
            "client_id": self.WEB_CLIENT_ID,
            "code": code,
            "redirect_uri": self.WEB_REDIRECT_URI
        }, catch_response=True) as first_response:

            if first_response.status_code != 200:
                first_response.failure(
                    f"Primer canje fallo inesperadamente con {first_response.status_code}"
                )
                return

            first_response.success()

        # Segundo canje del mismo code: deberia fallar (400 o 401)
        # Esto dispara auth_code_reuse en models.py -> consume_authorization_code()
        with self.client.post("/token", data={
            "grant_type": "authorization_code",
            "client_id": self.WEB_CLIENT_ID,
            "code": code,
            "redirect_uri": self.WEB_REDIRECT_URI
```

```
        }, catch_response=True) as second_response:

            if second_response.status_code in (400, 401):
                second_response.success() # Es el comportamiento esperado
            else:
                second_response.failure(
                    f"Se esperaba 400/401 en replay attack, pero llego {second_response.
                        status_code}"
                )

    @task(1)
    def revoked_token_usage_attack(self):
        """
        Obtiene un refresh token valido, lo revoca y luego intenta usarlo.
        Simula un ataque de token replay con un token ya revocado.

        Metricas activadas:
            - tokens_revoked{token_type="refresh"} / tokens_revoked{token_type="access"}
            - active_tokens (decrementos al revocar)
            - revoked_token_usage (al intentar usar el token revocado)
            - http_requests_token con status 400
        """
        # Paso 1: Obtenemos tokens validos
        code = self._get_authorization_code()
        if not code:
            return

        refresh_token = None
        with self.client.post("/token", data={
            "grant_type": "authorization_code",
            "client_id": self.WEB_CLIENT_ID,
            "code": code,
            "redirect_uri": self.WEB_REDIRECT_URI
        }, catch_response=True) as token_response:

            if token_response.status_code != 200:
                token_response.failure(
                    f"Obtencion de tokens para revocacion fallo con {token_response.
                        status_code}"
                )
                return

            try:
                tokens = token_response.json()
                refresh_token = tokens.get("refresh_token")
            except Exception:
                token_response.failure("No se pudo parsear la respuesta")
                return

            token_response.success()

        if not refresh_token:
            return
```

```
344
345         # Paso 2: Renovamos el token una vez para que el anterior quede revocado
346         # (muchos servidores revocan el refresh token viejo al emitir uno nuevo)
347         with self.client.post("/token", data={
348             "grant_type": "refresh_token",
349             "client_id": self.WEB_CLIENT_ID,
350             "refresh_token": refresh_token
351         }, catch_response=True) as rotate_response:
352
353             if rotate_response.status_code != 200:
354                 rotate_response.success() # No nos importa el resultado aqui
355                 return
356
357             rotate_response.success()
358
359         # Paso 3: Intentamos usar el refresh token ORIGINAL (ya revocado)
360         # Esto dispara revoked_token_usage en models.py -> validate_refresh_token()
361         with self.client.post("/token", data={
362             "grant_type": "refresh_token",
363             "client_id": self.WEB_CLIENT_ID,
364             "refresh_token": refresh_token
365         }, catch_response=True) as revoked_response:
366
367             if revoked_response.status_code in (400, 401):
368                 revoked_response.success() # Comportamiento esperado
369             else:
370                 revoked_response.failure(
371                     f"Se esperaba 400/401 al usar token revocado, pero llego {
                        revoked_response.status_code}"
372                 )
373
374     @task(1)
375     def invalid_refresh_token(self):
376         """
377         Intenta usar un refresh token completamente falso.
378         Esto prueba el error de refresh_token invalido (no existe en BD).
379
380         Metricas activadas:
381             - auth_flows_started / auth_flows_failed {grant_type="refresh_token"}
382             - http_requests / http_requests_token con status 400
383         """
384         with self.client.post("/token", data={
385             "grant_type": "refresh_token",
386             "client_id": self.VALID_CLIENT_ID,
387             "client_secret": self.VALID_CLIENT_SECRET,
388             "refresh_token": "refresh_token_completamente_falso_xyz123"
389         }, catch_response=True) as response:
390
391             if response.status_code == 400:
392                 response.success()
393             else:
394                 response.failure(
395                     f"Se esperaba 400 para refresh_token invalido, pero llego {response.
```

```
                    status_code}"
            )

    # ========================================================================
    # ENDPOINTS DE ADMINISTRACION
    # ========================================================================

    @task(1)
    def list_admin_clients(self):
        """
        Consulta el listado de clientes registrados en la aplicacion.

        Metricas activadas:
            - http_requests / http_requests_admin con status 200
            - requests_in_progress / request_duration para /admin/clients
        """
        response = self.client.get("/admin/clients")

        if response.status_code != 200:
            response.failure(f"/admin/clients GET fallo con {response.status_code}")

    # ========================================================================
    # ENDPOINT DE METRICAS
    # ========================================================================

    @task(1)
    def metrics_endpoint(self):
        """
        Comprueba que el endpoint /metrics sigue respondiendo bajo carga.
        Tiene poco peso porque Prometheus sera quien normalmente consulte este endpoint.

        Metricas activadas:
            - http_requests con endpoint="/metrics" y status 200
            - requests_in_progress / request_duration para /metrics
        """
        response = self.client.get("/metrics")

        if response.status_code != 200:
            response.failure(f"/metrics fallo con {response.status_code}")

    # ========================================================================
    # METODO AUXILIAR PRIVADO
    # ========================================================================

    def _get_authorization_code(self) -> str | None:
        """
        Realiza el paso de autorizacion (POST /authorize) con credenciales validas
        y devuelve el code extraido de la respuesta o de la cabecera Location.
        Devuelve None si no se pudo obtener el code.
        """
        with self.client.post("/authorize", data={
            "client_id": self.WEB_CLIENT_ID,
            "redirect_uri": self.WEB_REDIRECT_URI,
```

```
            "response_type": "code",
            "scope": "read",
            "state": "locust-internal",
            "username": self.VALID_USERNAME,
            "password": self.VALID_PASSWORD
        }, catch_response=True, allow_redirects=False) as response:

            if response.status_code not in (200, 302):
                response.failure(
                    f"_get_authorization_code fallo con {response.status_code}"
                )
                return None

            code = None

            # Intentamos extraer el code de la cabecera Location (redirect 302)
            location = response.headers.get("Location", "")
            if "code=" in location:
                code = location.split("code=")[1].split("&")[0]

            # Si no hay redirect, intentamos parsear el JSON del cuerpo
            if not code and response.status_code == 200:
                try:
                    data = response.json()
                    code = data.get("code")
                except Exception:
                    pass

            response.success()
            return code
```

La cuál presentaba esta lista de problemas Problema 1 — brute_force_user_login nunca llegaba a comprobar el usuario Cuando Locust mandaba la petición al POST /authorize, la app miraba primero si venían todos los campos del formulario. Como faltaban action, scope, state y response_type, la app fallaba antes de llegar a comprobar el usuario y la contraseña. Por eso user_auth_failed nunca se incrementaba. Es como si intentaras entrar a un edificio pero antes de llegar a la puerta te para un guardia porque te falta el pase de visitante. Nunca llegas a intentar abrir la puerta. Problema 1 — brute_force_use_login nunca llegaba a comprobar el usuario Cuando Locust mandaba la petición al POST /authorize, la app miraba primero si venían todos los campos del formulario. Como faltaban action, scope, state y response_type, la app fallaba antes de llegar a comprobar el usuario y la contraseña. Por eso user_auth_failed nunca se incrementaba. Es como si intentaras entrar a un edificio pero antes de llegar a la puerta te para un guardia porque te falta el pase de visitante. Nunca llegas a intentar abrir la puerta. Problema 3 — Nunca probabas revoked_token_usage Esta es una de tus métricas EDR más importantes pero nunca la ejercitabas. Sin una tarea que intente usar un token inválido como refresh token, esa métrica siempre valdría cero en Grafana durante las pruebas de carga.

```
"""
app.py --Flask application entry point.
```

```
3
4  Wires together the database, blueprints, and configuration.
5  Run with: uv run python app.py
6  """
7  #DISCLAIMER we've commented in english to mantain the consistence of the code
8
9  #request had the request info
10 #g will be use to store the request live time
11 from flask import Flask, request, g
12 #wsi will be the way promethus get their data
13 from prometheus_client import make_wsgi_app
14 #use for the comunication btw the wsgi and metrics.py
15 from werkzeug.middleware.dispatcher import DispatcherMiddleware
16 import time
17
18 from database import close_db, init_db
19 from oauth2_routes import oauth2
20 from admin_routes import admin
21 #we need this lib for the metrics that we've allready defined on metrics.py
22 from metrics import (
23        http_requests,
24        http_requests_token,
25        http_requests_admin,
26        http_requests_authorize,
27        request_duration,
28        requests_in_progress,
29        unknown_client_attempts
30 )
31
32 from werkzeug.serving import run_simple
33 # ---------------------------------------------------------------------------
34 # Hardcoded test users (resource owners for the authorization code flow).
35 # In production these would live in a proper user store.
36 # ---------------------------------------------------------------------------
37 USERS = {
38     "testuser": "testpass",
39     "admin": "adminpass",
40 }
41
42
43 def create_app() -> Flask:
44     app = Flask(__name__)
45     app.secret_key = "dev-secret-key-change-in-production"
46
47     # Store users in app config so routes can access them
48     app.config["USERS"] = USERS
49
50     # Register blueprints
51     app.register_blueprint(oauth2)
52     app.register_blueprint(admin)
53
54     # Database lifecycle
55     app.teardown_appcontext(close_db)
```

```
56
57      # Ensure tables exist on first launch
58      with app.app_context():
59          init_db()
60
61      #Automatic HTTP metrics log
62      # the Flask decorator
63      @app.before_request
64      def before_request():
65          # Store the moment of the request arrives
66          g.start_time = time.time()
67          requests_in_progress.labels(endpoint=request.path).inc()
68      #This force that every request must execute this methods
69      @app.after_request
70      def after_request(response):
71          # get the request endpoint
72          endpoint = request.path
73
74          # we increment the general counter
75          http_requests.labels(
76              method=request.method,
77              endpoint=endpoint,
78              status_code=str(response.status_code)
79          ).inc()
80
81          # We increment the specific counter for each endpoint
82          if "/token" in endpoint:
83              http_requests_token.labels(
84                  method=request.method,
85                  status_code=str(response.status_code)
86              ).inc()
87          elif "/authorize" in endpoint:
88              http_requests_authorize.labels(
89                  method=request.method,
90                  status_code=str(response.status_code)
91              ).inc()
92          elif "/admin" in endpoint:
93              http_requests_admin.labels(
94                  method=request.method,
95                  status_code=str(response.status_code)
96              ).inc()
97          requests_in_progress.labels(endpoint=request.path).dec()
98          # Calc and store the request duration
99          duration = time.time() - g.start_time
100         request_duration.labels(
101             endpoint=request.path,
102             method=request.method
103         ).observe(duration)
104         return response
105
106     return app
107
108
```

```python
if __name__ == "__main__":
    # Cretaion of the Flask app
    flask_app = create_app()

    # Merge Flask with the /metrics (Prometheus) endpoint
    application = DispatcherMiddleware(flask_app, {
        "/metrics": make_wsgi_app()
    })

    print("OAuth2 server running on http://localhost:5001")
    print("Metrics available on http://localhost:5001/metrics")
    print("Endpoints:")
    print(" GET /authorize --Authorization Code flow (browser)")
    print(" POST /token --Token endpoint")
    print(" GET /admin/clients --List all clients")
    print(" POST /admin/clients --Register a new client")
    print(" DELETE /admin/clients/<id> --Delete a client")
    print()

    run_simple("0.0.0.0", 5001, application, use_reloader=True, use_debugger=True)
```

Práctica 2 — Parte II

Instrumentación EDR, Modificaciones y Locust

---

8 Enfoque EDR del sistema de métricas

8.1 Motivación

El enunciado de la práctica señala explícitamente que uno de los objetivos de las pruebas de carga es identificar *"si surgen fallos de seguridad fruto del alto nivel de carga"* y que *"se podrán añadir métricas adicionales"*. Partiendo de este punto, se ha optado por un enfoque orientado a la seguridad, inspirado en los sistemas EDR (*Endpoint Detection and Response*), que permite detectar comportamientos anómalos en tiempo real durante las pruebas de carga.

Un sistema EDR monitoriza eventos de seguridad en tiempo real para detectar patrones de ataque como fuerza bruta, reutilización de tokens o intentos de acceso con credenciales comprometidas. Prometheus y Grafana permiten replicar este comportamiento de forma sencilla sobre el servidor OAuth2.

8.2 Métricas adicionales implementadas

Se han añadido tres categorías de métricas adicionales sobre las requeridas por el enunciado:

8.2.1. Categoría 1 — Rendimiento del sistema

Métrica	Descripción
`oauth2_request_duration_seconds`	Histogram del tiempo de respuesta por endpoint y método HTTP. Permite calcular el percentil 95 que el enunciado fija en 250ms como umbral de referencia.
`oauth2_requests_in_progress`	Gauge de peticiones HTTP simultáneas en curso en este momento. Útil para detectar ataques DoS bajo pruebas de estrés.

Cuadro 3: Métricas de rendimiento

8.2.2. Categoría 2 — Estado de tokens

Métrica	Descripción
`oauth2_active_tokens`	Gauge de tokens activos en el sistema por tipo. Sube al emitir tokens y baja al revocarlos.
`oauth2_tokens_revoked_total`	Counter de tokens revocados por tipo. Refleja la rotación de tokens en el flujo refresh_token.
`oauth2_pending_auth_codes`	Gauge de códigos de autorización emitidos pero no canjeados todavía. Un valor creciente indica clientes que piden códigos sin completar el flujo.
`oauth2_revoked_token_usage_total`	Counter de intentos de uso de tokens ya revocados. Señal de posible token replay attack.
`oauth2_expired_token_usage_total`	Counter de intentos de uso de tokens expirados.
`oauth2_auth_code_reuse_total`	Counter de intentos de reutilización de códigos de autorización ya consumidos. Señal de authorization code replay attack.

Cuadro 4: Métricas de estado de tokens

8.2.3. Categoría 3 — Detección de ataques

Métrica	Descripción
`oauth2_brute_force_attempts_total`	Counter de intentos fallidos de autenticación por client_id. Detecta ataques de fuerza bruta M2M.
`oauth2_user_auth_failed_total`	Counter de intentos fallidos de login por username y client_id. Detecta ataques de fuerza bruta sobre usuarios.

Cuadro 5: Métricas de detección de ataques

9 Modificaciones a metrics.py

El fichero `metrics.py` centraliza la definición de todos los contadores. Se han añadido los tres tipos de métricas de Prometheus: `Counter`, `Histogram` y `Gauge`.

El `Histogram` requiere la definición explícita de *buckets*, que son los intervalos de tiempo en los que se agrupan las mediciones. Se han elegido buckets optimizados para una aplicación web, incluyendo el valor `0.25` que corresponde al umbral de 250ms mencionado en el enunciado:

```
        request_duration = Histogram(
        "oauth2_request_duration_seconds",
        "Tiempo de respuesta de cada peticion",
        ["endpoint", "method"],
        # 0.25 es el umbral de 250ms que marca el enunciado
        buckets=[0.01, 0.05, 0.1, 0.25, 0.5, 1.0, 2.5, 5.0]
        )
```

Listing 3: Histogram con buckets para web

Una decisión de diseño importante fue no añadir `client_id` como etiqueta en `tokens_issued`. Aunque inicialmente se contempló para tener información por cliente, se descartó porque Prometheus crea una serie temporal por cada combinación única de etiquetas. Con una etiqueta de alta cardinalidad como `client_id` el número de series crecería proporcionalmente al número de clientes, lo que en una VM con 2GB de RAM podría causar problemas de memoria durante las pruebas de carga.

10 Modificaciones a models.py

`models.py` es la capa de acceso a datos. Se ha instrumentado con métricas en cuatro funciones:

10.1 store_authorization_code

Se incrementa `pending_auth_codes` cada vez que se genera un código de autorización, reflejando que existe un código emitido pero no canjeado:

```
        def store_authorization_code(...) -> str:
        # ... codigo original ...
        db.commit()
        # A new code has been issued but not yet consumed
        pending_auth_codes.inc()
        return code
```

Listing 4: Instrumentacion de store_authorization_code

10.2 consume_authorization_code

Se decrementa `pending_auth_codes` cuando el código se consume correctamente. Además se detectan dos casos de seguridad relevantes: el intento de reutilización de un código ya usado (`auth_code_reuse`) y el intento de uso de un código expirado:

```
        def consume_authorization_code(code: str) -> Optional[dict]:
        # ... codigo original ...
        if data["used"]:
        # Someone is trying to reuse an already consumed code
        # This is a sign of an authorization code replay attack
        auth_code_reuse.inc()
        return None

        if time.time() > data["expires_at"]:
        return None

        db.execute("UPDATE authorization_codes SET used = 1 ...")
        db.commit()
        # Code consumed correctly, no longer pending
        pending_auth_codes.dec()
        return data
```

Listing 5: Instrumentacion de consume_authorization_code

10.3 issue_token_pair

Se incrementan los gauges de tokens activos cada vez que se emiten tokens. Se distingue entre access tokens y refresh tokens:

```
        def issue_token_pair(...) -> dict:
        # ... codigo original ...
        db.commit()
        # Track active tokens in the system
        active_tokens.labels(token_type="access").inc()
        if refresh_token:
        active_tokens.labels(token_type="refresh").inc()
        return response
```

Listing 6: Instrumentacion de issue_token_pair

10.4 validate_refresh_token

Se detectan los dos casos de fallo relevantes para la seguridad: el intento de uso de un token revocado y el intento de uso de un token expirado. Es importante que estos contadores se incrementen aquí en `models.py` y no en `oauth2_routes.py`, porque esta función tiene acceso directo al motivo exacto del fallo:

```
        def validate_refresh_token(token: str) -> Optional[dict]:
        # ... codigo original ...
        if data["revoked"]:
        # Someone is using a token after it has been revoked
        # This is a sign of a token replay attack
        revoked_token_usage.inc()
        return None

        if time.time() > data["expires_at"]:
```

```
        # Someone is trying to use an expired token
        expired_token_usage.inc()
        return None

        return data
```

Listing 7: Instrumentacion de validate_refresh_token

10.5 revoke_refresh_token

Se incrementan los counters de tokens revocados y se decrementan los gauges de tokens activos. La rotación de tokens del flujo `refresh_token` revoca tanto el refresh token antiguo como su access token asociado:

```
        def revoke_refresh_token(token: str) -> None:
        # ... codigo original ...
        db.commit()
        # Both the refresh token and its associated access token are revoked
        tokens_revoked.labels(token_type="refresh").inc()
        tokens_revoked.labels(token_type="access").inc()
        # Decrement active tokens accordingly
        active_tokens.labels(token_type="refresh").dec()
        active_tokens.labels(token_type="access").dec()
```

Listing 8: Instrumentacion de revoke_refresh_token

11 Modificaciones a app.py

Se han añadido dos instrumentaciones importantes en `app.py` mediante decoradores Flask.

11.1 Exposición del endpoint /metrics

Se utiliza `DispatcherMiddleware` para combinar la aplicación Flask con la mini aplicación de Prometheus que genera el endpoint `/metrics`. Esto permite que Prometheus pueda leer las métricas en `http://localhost:5001/metrics`:

```
        from prometheus_client import make_wsgi_app
        from werkzeug.middleware.dispatcher import DispatcherMiddleware
        from werkzeug.serving import run_simple

        if __name__ == "__main__":
        flask_app = create_app()
        # Combine Flask with the Prometheus /metrics endpoint
        application = DispatcherMiddleware(flask_app, {
            "/metrics": make_wsgi_app()
        })
        run_simple("0.0.0.0", 5001, application,
        use_reloader=True, use_debugger=True)
```

Listing 9: Exposicion del endpoint /metrics

11.2 Registro automático de métricas HTTP

Se utilizan los decoradores `@before_request` y `@after_request` de Flask para registrar automáticamente todas las peticiones HTTP sin modificar cada ruta individualmente. Esto sigue el principio DRY (*Don't Repeat Yourself*):

```
        @app.before_request
        def before_request():
        # Store arrival time and increment in-progress counter
        g.start_time = time.time()
        requests_in_progress.labels(endpoint=request.path).inc()

        @app.after_request
        def after_request(response):
        endpoint = request.path

        # General HTTP counter
        http_requests.labels(
        method=request.method,
        endpoint=endpoint,
        status_code=str(response.status_code)
        ).inc()

        # Specific counters per endpoint
        if "/token" in endpoint:
        http_requests_token.labels(
        method=request.method,
        status_code=str(response.status_code)
        ).inc()
        elif "/authorize" in endpoint:
        http_requests_authorize.labels(
        method=request.method,
        status_code=str(response.status_code)
        ).inc()
        elif "/admin" in endpoint:
        http_requests_admin.labels(
        method=request.method,
        status_code=str(response.status_code)
        ).inc()

        # Decrement in-progress and record duration
        requests_in_progress.labels(endpoint=request.path).dec()
        duration = time.time() - g.start_time
        request_duration.labels(
        endpoint=request.path,
        method=request.method
        ).observe(duration)

        return response
```

Listing 10: Decoradores before y after request

El decorador `@before_request` se ejecuta antes de procesar cada petición. Se guarda el instante de llegada en `g.start_time` y se incrementa el gauge de peticiones en curso.

El decorador `@after_request` se ejecuta después de procesar cada petición, cuando ya se conoce el código de respuesta. Se incrementan los counters HTTP, se decrementa el gauge de peticiones en curso y se registra la duración en el Histogram mediante `observe()`.

12 Pruebas de carga con Locust

12.1 Instalación

Locust se instala como una dependencia del proyecto mediante `uv`:

```
cd ~/oauth2
uv add locust
```

Se arranca con el siguiente comando, que lanza la interfaz web en el puerto 8089:

```
uv run locust -f locustfile.py --host=http://localhost:5001
```

12.2 Diseño del locustfile

El fichero `locustfile.py` define una clase de usuario con cinco tareas con pesos distintos. Los pesos se han elegido para simular un escenario realista donde el 50 % del tráfico es legítimo y el 50 % son ataques:

Tarea	Tipo	Peso	Probabilidad
valid_client_credentials	Tráfico legítimo M2M	4	40 %
brute_force_api	Ataque fuerza bruta M2M	2	20 %
brute_force_user_login	Ataque fuerza bruta web	2	20 %
valid_user_login	Tráfico legítimo web	1	10 %
replay_attack	Ataque token replay	1	10 %

Cuadro 6: Tareas de Locust con sus pesos y probabilidades

12.3 Descripción de cada tarea

valid_client_credentials — Simula el tráfico normal de una aplicación M2M que pide tokens correctamente. Es la tarea más frecuente porque en producción la mayoría del tráfico es legítimo. Dispara las métricas de `auth_flows_success` y `access_tokens_issued`.

brute_force_api — Simula un atacante que intenta adivinar el `client_secret` mediante fuerza bruta. Envía credenciales incorrectas intencionadamente y espera un 401 como respuesta. Dispara `brute_force_attempts` y `auth_flows_failed`.

brute_force_user_login — Simula un atacante que intenta adivinar la contraseña de un usuario mediante fuerza bruta a través del formulario de login. Dispara `user_auth_failed`. Es importante incluir todos los campos del formulario (`action`, `response_type`, `scope` y `state`) para que la app llegue a comprobar las credenciales.

valid_user_login — Simula un usuario legítimo haciendo login correctamente. Sin esta tarea las métricas de `authorization_code` exitoso siempre estarían a cero durante las pruebas, lo que no reflejaría la realidad.

replay_attack — Simula un atacante que intenta reutilizar un token de acceso como si fuera un refresh token. Este escenario replica un ataque de tipo token replay donde el atacante ha robado un token e intenta usarlo fuera de su contexto. Dispara `revoked_token_usage`.

12.4 Configuración de las pruebas

Desde la interfaz web de Locust en `http://IP-DE-LA-VM:8089` se configuran dos pruebas:

Prueba de carga — Evalúa el comportamiento del sistema en condiciones normales:

Parámetro	Valor
Number of users	50
Spawn rate	5 usuarios/segundo
Duración	5 minutos

Cuadro 7: Configuración de la prueba de carga

Prueba de estrés — Evalúa el comportamiento del sistema en condiciones extremas para encontrar el límite real:

Parámetro	Valor
Number of users	200
Spawn rate	10 usuarios/segundo
Duración	3 minutos

Cuadro 8: Configuración de la prueba de estrés

12.5 Protección de la VM durante las pruebas

Dado que la VM dispone de recursos limitados (4GB de disco y 2GB de RAM), se han tomado las siguientes medidas para evitar que las pruebas de carga agoten los recursos:

Prometheus se arranca con límites explícitos de retención de datos:

```
./prometheus --config.file=prometheus.yml \
--storage.tsdb.retention.time=2h \
--storage.tsdb.retention.size=500MB
```

Se monitoriza el uso de recursos en tiempo real durante las pruebas con:

```
watch -n 2 'echo "=== DISCO ===" && df -h / && \
echo "=== RAM ===" && free -h && \
echo "=== CPU ===" && top -bn1 | head -5'
```

El orden correcto de arranque es el siguiente para garantizar que todos los componentes están disponibles antes de iniciar las pruebas:

1. Terminal 1 — `uv run python app.py`
2. Terminal 2 — `./prometheus -config.file=prometheus.yml`
3. Terminal 3 — `uv run locust -f locustfile.py -host=http://localhost:5001`
4. Terminal 4 — `watch` para vigilancia de recursos

Y el orden de parada es el inverso, esperando 30 segundos entre Locust y Prometheus para que este último recoja los últimos datos antes de apagarse.

13 Verificación manual de métricas

Antes de ejecutar Locust se ha desarrollado un script de verificación `test_metrics.py` que comprueba cada métrica de forma individual y automática. El script ejecuta nueve tests en orden:

Test	Nombre	Métricas verificadas
1	client_credentials válido	auth_flows_success, active_tokens
2	Fuerza bruta M2M	auth_flows_failed, brute_force_attempts
3	Fuerza bruta web	user_auth_failed
4	authorization_code completo	refresh_tokens_issued, pending_auth_codes
5	refresh_token válido	tokens_revoked, active_tokens
6	Token replay attack	revoked_token_usage
7	Auth code reuse	auth_code_reuse
8	Peticiones en paralelo	requests_in_progress, request_duration
9	Token falso/expirado	expired_token_usage

Cuadro 9: Tests del script de verificación manual

Se ejecuta con:

```
cd ~/oauth2
uv run python test_metrics.py
```

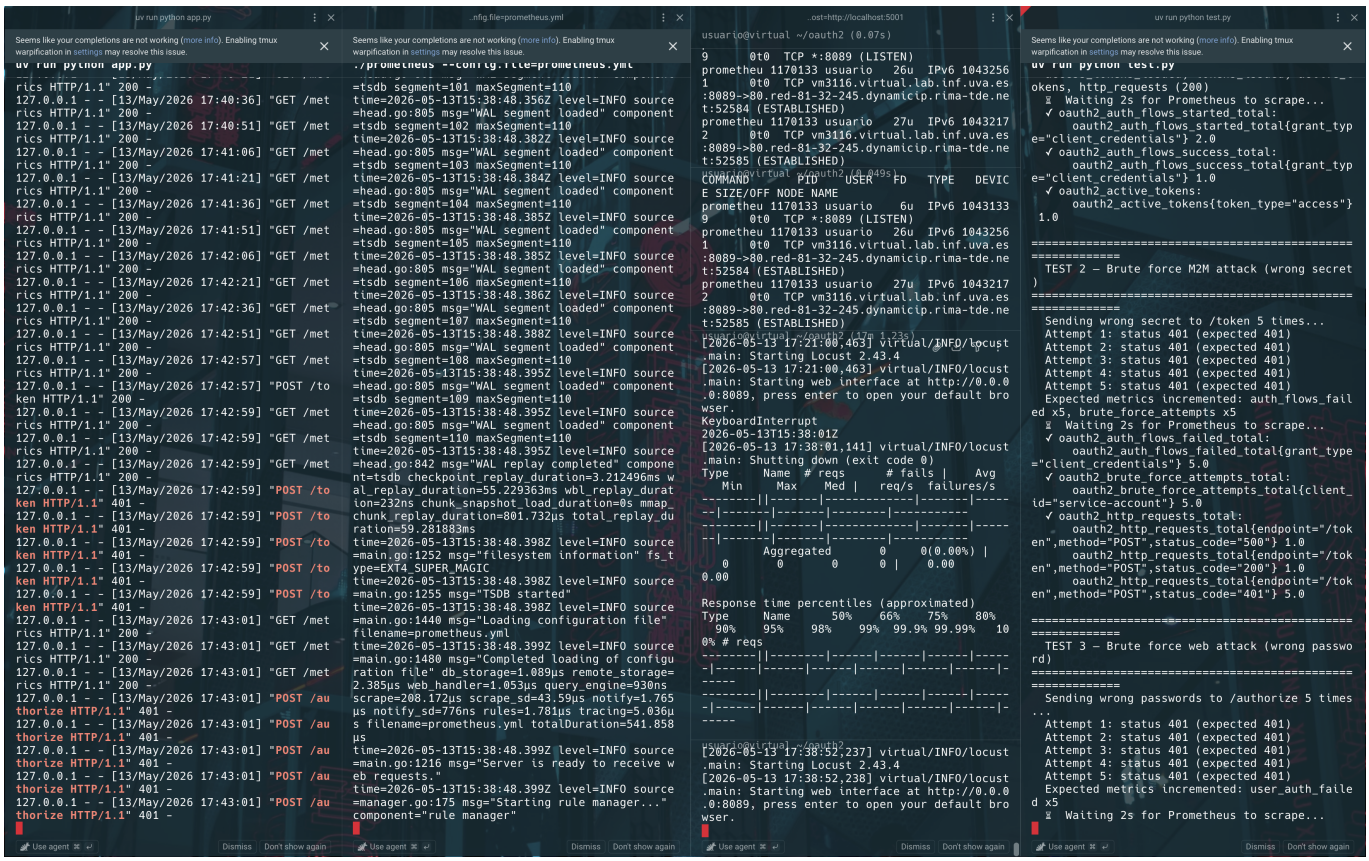


Figura 1: Ejecutamos la *app.py*, el locust y el prometheus, con el error de ejecución en los puertos erróneos

14 Grafana

Al igual que en la anterior entrega, hemos tenido que hacer un *dashboard* en grafana para tener una reperesentación gráfica de nuestro trabajo en esta práctica



Figura 2: Nuestro panel de grafana para esta práctica. Aunque entraremos más en detalle, nuestro *dashboard* presenta dos tipos de metricas, las clásicas de pruebas de carga y las métricas específicas para nuestro enfoque EDR

Una vez que ya tenemos una idea de cómo hemos ditribuido el *dashboard* vamos a ir panel a panel explicándo un poco la intención y alguna peculiaridad de la representación.
Las consultas con Prometheus son mucho más simples que las que nos tocó hacer con Postgres en la práctica anterior, pero al ser una herramienta nueva, nos ha llevado nuestro tiempo acostumbrando a su sintaxis y sus operandos, teniendo que pedir consejo a Adrián sobre como escribir las peticiones para poder reflejar de una manera adecuada los datos.

14.1 Ataques de fuerza burta vs *credential stuffing*

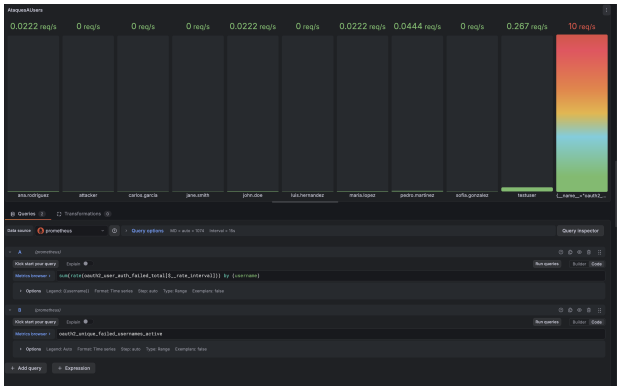


Figura 3: Con este panel estamos intentando diferenciar entre los errores de un usuario al escribir su contraseña con un posible ataque de fuerza bruta para entrar a la cuenta de un usuario.

Esta métrica pertenece a nuestro enfoque EDR, pero para entenderla bien, vamos a explicar cuáles son los dos tipos de ataques que estamos midiendo. Un *Brute Forze Attack* se trata de una técnica de hacking que se basa en probar el acceso a la cuenta de un usuario empleando un diccionario de contraseña, mientras que el *credential stuffing* se basa en intentar *loggearse* en un sistema empleando contraseñas de otros sitios de ese mismo usuario/mail.
Este panel muestra qué usuarios han sido atacados durante las pruebas, aquellos con más intentos fallidos son víctimas de fuerza bruta mientras que la presencia de muchos usuarios distintos con pocos intentos cada uno indica *credential stuffing*.

14.2 Ratio de Tokens

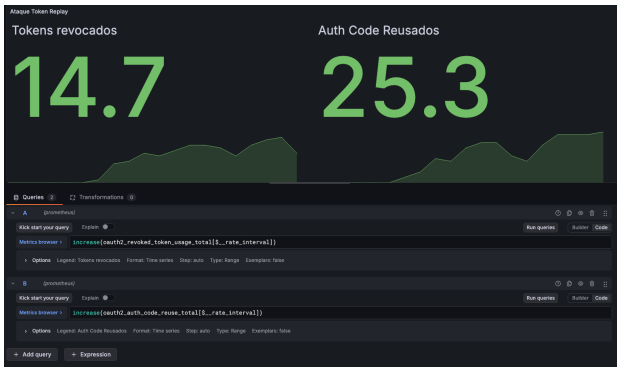


Figura 4: Tipos de tokens que maneja nuestro sistema

Con este panel lo que estamos intentando mirar es si estamos sufriendo ataques del tipo *Token Replay*, los cuales se basan en que un atacante ha conseguido un token válido y lo está empleando desde su cuenta para poder realizar ataques sobre nuestro sistema.
Para tener una visión limpia sobre este tipo de ataques estamos usando las fuunciones de incremento absoluto (*increase()*) en PromQL, las cuales revelan el número exacto de intentos de ataque neutralizados durante la ventana de evaluación. Además empleamos la métrica `oauth2_revoked_token_usage_total` para contabilizar las peticiones bloqueadas que intentan operar utilizando un *Refresh Token* que ya ha sido invalidado. ya que cualquier intento posterior de usarlas no solo es denegado, sino que queda registrado en este contador como una brecha de seguridad interceptada.

14.3 Fallos del login de los usuarios



Figura 5: Número de usuarios de nuestro sistema que han tenido fallos en su login

A simple vista, esta métrica parece un poco irrelevante ¿qué información útil aporta el número de usuarios que se han equivocado al introducir sus datos al hacer el *login*?, y la respuesta a esta pregunta es simple, un alto número de usuarios que fallan en su inicio de sesión es un indicativo de que algo anda mal, de tal manera que unido al panel anterior, nos da la información de que puede que uno de esos usuarios esté sufiendo un ataque y que por ende sea un vector de infección a nuestro sistema.

14.4 Fallos del login de los usuarios



Figura 6: Usuarios de nuestro sistema que han tenido fallos en su login

Métrica complementaria a la anterior, la cuál ya no sólo nos dice los intentos fallidos de *login en el sistema*, sino que tenemos una visión de qué usuarios están teniendo este comportamiento.
Decimos que es complementaria puesto que si el adminitrador del sistema detecta en el panel anterior un número grande de errores de login por parte de usuarios, consultado este panel puede ver qué usuario en concreto está teniendo este comportamiento, lo que permite contactar con él para que revise la actividad o en su defecto, cortarle el acceso al sistema por si te trata de un bot o un usuario que ha perdido el acceso a sus credenciales.
Así como dato extra a gráfica se construye mediante una consulta PromQL que calcula la tasa de fallos (*rate()*) y los agrupa por nombre de usuario (sum(...) by (username)), para poder realizar esta distinción.

14.5 Logins correctos en el sistema.



Figura 7: Número de logins correctos en el sistema

Esta métrica es una clásica en lo que se refiere al monitoreo de un sistema y a las pruebas de carga, con locust estamos testeando una carga de usuarios y peticiones simultáneas de tal manera que este panel nos sirve para ver cuántos usuarios están intentando acceder a nuestro sistema.

14.6 Peticiones a los diferentes endpoints de nuestro sistema

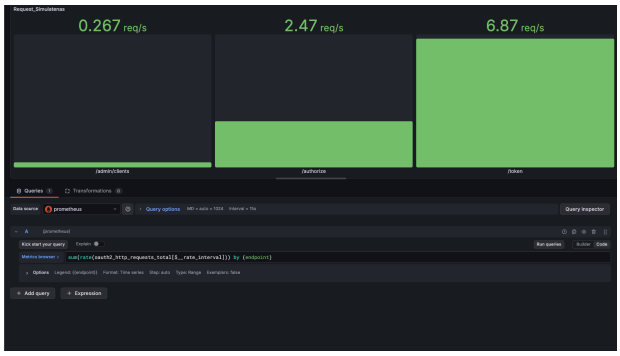


Figura 8: Peticiones que le llegan a los diferentes endpoints al sistema.

En este panel, nos encontramos con la distribución de peticiones a nuestro sistema.
Nos puede llamar la atención la distribución tan dispar de peticiones a cada uno de los *endpoints* pero hay que tener en cuenta que (`authorize`) es el endpoint que refleja una interacción humana, es decir, sería el formulario que seguiría un cliente al autenticarte en el sistema, por lo que, siguiendo el consejo de Adrián hemos generado una carga de trabajo para ello.
Sin embargo el `/admin/clients` es la ruta de administración interna, la cual no debería estar expuesto a ataques del exterior, por lo que registra un tráfico residual y estable.
Hemos decidido implementar este panel para tener una información de la carga de trabajo en nuestro sistema, ya que podemos modificar los recursos destinados a cada uno o en caso de tener que tomar medidas saber exactamente la parte de nuestro sistema que está lastrando la *performance* del entorno.

14.7 Distribución de las peticiones en nuestro sistema

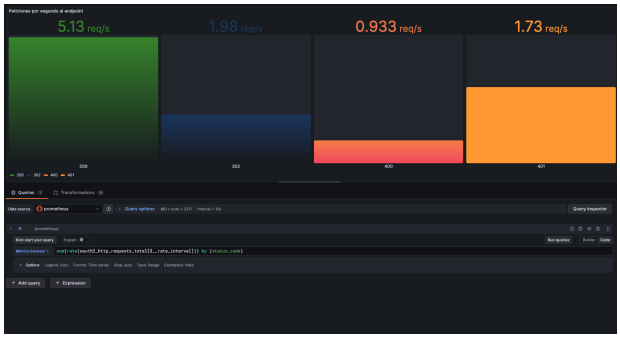


Figura 9: Distribución de las peticiones en nuestro sistema.

Panel que sirve para ver el estado de sistemas, el 200 es el código para las peticiones correcta por eso se representa de color ver mientras que las peticiones en el sector de color amarillo agrupan todas las respuestas que no han resultado en un éxito, principalmente errores 400 de *Bad Request*, 401 de *Unauthorized* y posibles redirecciones 302., los cuales nos aportan una información muy relevante ya que los errores pueden implicar un mal funcionamiento del sistema.

14.8 Ratio peticiones correctas fallidas por los usuarios



Figura 10: Tipos de tokens que maneja nuestro sistema

Nuestra idea con este panel es ver cómo se comportaba nuestro sistema ante los flujos de autenticación, como ya hemos comentado, nuestras pruebas no sólo buscan una prueba de carga sino comproabr la seguridad del sistema por lo que esté gráfico nos dice que el sistema es capaz de mantener un fijo constante de autorizaciones válidas pese a todo el ruido de peticiones malicosas que le estamos haciendo, teniendo un comprotamiento estable por lo que se puede ver en la linea de tendencias y que además, no está dando falsos positivos, es decir estamos ante un sistema robusto en lo referente a los *logeos*.

14.8.1. Tiempo de respuesta a cada *endpint*

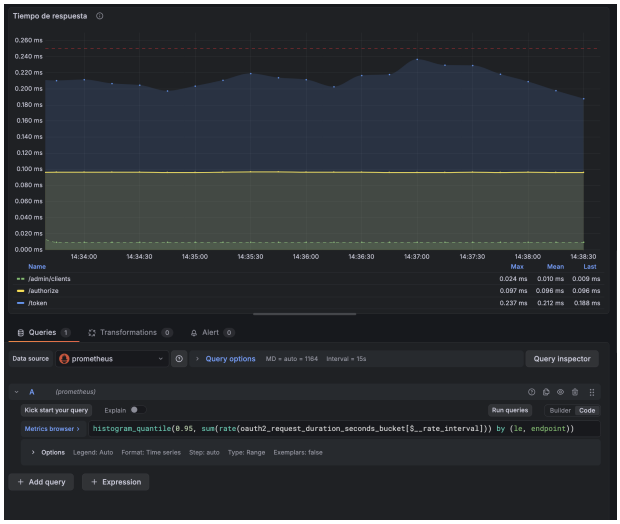


Figura 11: Gráfico que mietras el tiempo de respuesta a cada endpoint con una leyenda la cual nos da información exacta sobre los valores límite

Esta gráfica es de vita importancia para saber si los *endpoints* están funcionando como deberían, además de darnos la información de los tiempos de respuesta en función de la carga de trabajo que nosotros le estamos haciendo con el locuts.
Tenemos los 3 puntos , cada uno de un color, *token* (línea azul) es el punto de máximo trabajo de nuestra aplicación puesto que se encarga de la gestión de token y los registros de audotia, *authorized* (línea amarilla) es en el que se reflejan la respuesta de los usuarios por lo que la respuesta es casi instantánea mientras que el de *admin* al ser un *endpoint* interno tiene un tiempo de respues mínimo ya que no está expuesto a la interacción con los usuarios.

14.8.2. Tipos de *tokens* en nuestro sistema.



Figura 12: Información sobre todos los *tokens* emitidos en nuestro sistema.

Con este panel estamos registrando los tokens que se están emitiendo/empleando en nuestro sistema, debemos recordar que estamos empleando un servidor oauth2 para la autenticación por lo que esta estadistica es necesaria para comprobar que el sistema funciona bien y que no estamos sufriendo ningún tipo de ataque.
Como tenemos 3 tipos vamos a explicar cada uno de ellos

- **Query A (*Access Tokens*):** representa el número de sesiones activas que pueden realizar peticiones a recursos protegidos.
- **Query B (*Refresh Tokens*):** representa el número de sesiones de larga duración que permiten a los clientes obtener nuevas credenciales sin intervención del usuario.
- **Query C (*Total Tokens*):** como su nombre indica la suma de las anteriores.

14.8.3. Distribución de *tokens* en el sistema

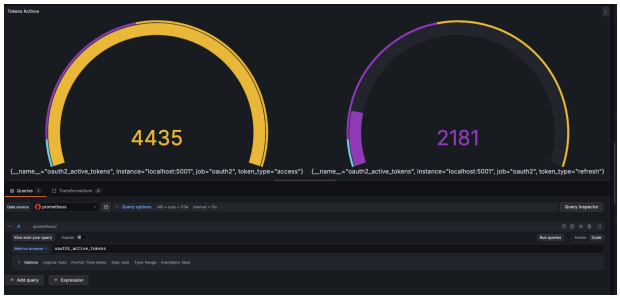


Figura 13: Información sobre todos los *tokens* emitidos en nuestro sistema.

Esta métrica es un histórico de todos los *tokens* emitidos en el sistema a nivel de acumulador, si nos damos cuanta esta métrica y la anterior tienen una relación, ya que una nos sirve de acumulador mientras que la anterior nos da una información de la producción de token y los tipos de cada uno.

14.8.4. Intentos de ataque de fuerza bruta

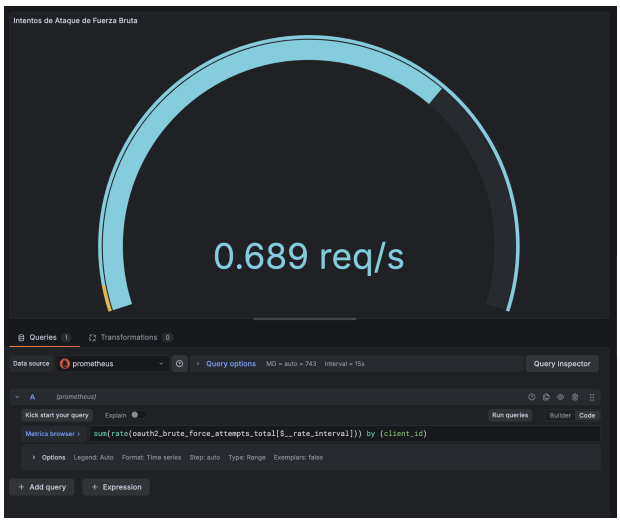


Figura 14: Peticiones consideradas de fuerza bruta que está recibiendo el sistema.

Esta métrica es vital para el enfoque EDR, puesto que nos da una información sobre un posible ataque al sistema. Nuestro enfoque ha sido el siguiente, tenemos una métrica que se incrementa cuando un usuario falla el login, debido a las condiciones del sistema no podemos tener un indicador más grande a puro código, pero si nos plantamos en el grafana, tenemos un panel para los intentos fallidos de login por usuario, lo que unido a un valor elevado de las *requests per second* en este panel nos permite aislar a aqellos usuarios que son probelmáticos.

14.8.5. Flujos de autenticació en cada uno de los *endpoints*

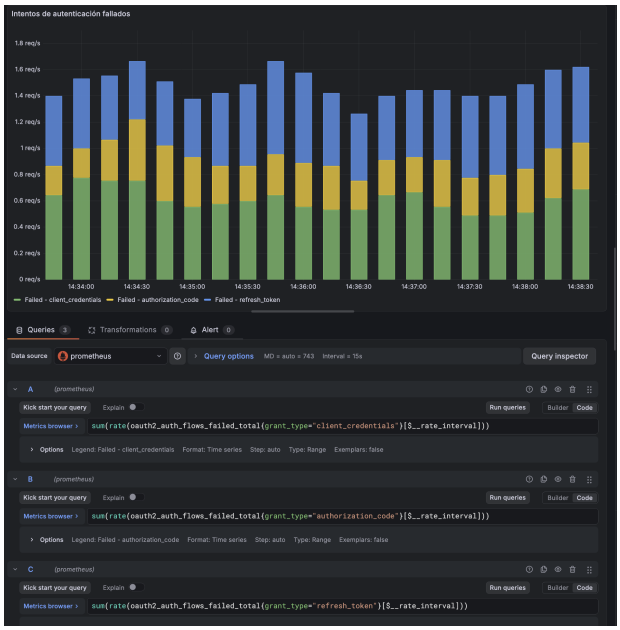


Figura 15: Tipos de tokens que maneja nuestro sistema

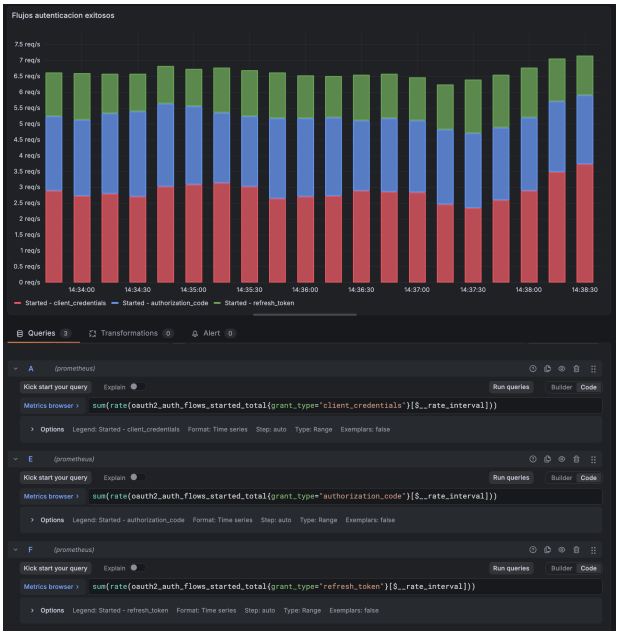


Figura 16: Tipos de tokens que maneja nuestro sistema

Para este panel lo que hemos buscado tener una representación visual de los intentos fallidos de inicio de sesión en cada uno de los *endpoints de nuestro sistema*

- **Verde**: registra fallos en la autenticación máquina-a-máquina. Una tasa alta aquí indica intentos de fuerza bruta contra secretos de aplicaciones o cuentas de servicio.
- **Amarillo**:representa intentos fallidos de inicio de sesión en el formulario web, usualmente vinculados a ataques de diccionario o *credential stuffing*.
- **Azul:** coontabiliza intentos de usar tokens de refresco inválidos, caducados o revocados. Es un indicador clave de ataques de *Replay* o de sesiones persistentes comprometidas.